

01：先理解整个项目

1. 项目面对的现实任务

假设有一个地图服务。用户不断提出这样的请求：

从道路节点 A 到道路节点 B，沿道路行驶的最短距离是多少？

一次请求叫作一次**最短路查询**。如果只有几条道路，直接搜索很快；但 Porto 实验道路图包含 133,839 个节点和 221,589 条有向边，查询数量有 98,082 条。面对大量重复查询，每次都在完整道路图上从头搜索，会反复做很多相似工作。

项目因此提出一个预处理思路：

1. 在查询真正到来前，先找出一些值得压缩的道路区域；
2. 离线计算这些区域的精确“穿越代价”；
3. 在线查询时，不再逐个搜索区域里的所有小道路节点，而是通过预先算好的 shortcut

快速穿过区域；

1. 无论是否使用压缩，最终最短距离必须完全相同。

项目真正研究的难点不是“能否压缩”，因为传统阶段已经证明能压缩；真正难点是：

在索引空间有限时，究竟应该压缩哪些区域，才能服务未来的查询需求？

2. 什么是“图”

这里的“图”不是图片，而是一种数学数据结构。一个图由两类对象组成：

- **节点 (node, 也叫 vertex)**：一个离散位置；
- **边 (edge)**：两个节点之间允许发生的连接。

在道路图里，可以粗略理解为：

- 节点代表路口、道路形状变化点或其他 OSM 路网点；
- 边代表从一个道路节点行驶到另一个节点的一段道路。

例如：

```
A --100m--> B --80m--> C
 \                ^
  \-----250m-----/
```

这里有三个节点 A、B、C。A 到 B 的边长 100 米，B 到 C 的边长 80 米，A 到 C 还有一条 250 米的边。A 到 C 的最短距离不是 250 米，而是 $100 + 80 = 180$ 米。

3. 什么是“有向图”

如果一条边只能按一个方向使用，这个图就是**有向图**。

道路必须考虑方向，因为现实中有：

- 单行道；
- 只允许驶入、不允许从相同方向驶出的连接；
- A 到 B 可行，但 B 到 A 不一定存在对应道路。

因此 $A \rightarrow B$ 和 $B \rightarrow A$ 是两条不同的边。代码同时保存：

- `adjacency`：从当前节点可以走到哪里；
- `reverse_adjacency`：哪些节点可以走到当前节点。

它们定义在 `src/graph_types.py` 的 `WeightedDiGraph` 中。

4. 什么是“带权图”

边上的数字叫作**权重（weight）**。本项目当前使用道路长度作为权重，所以最短路径

总道路长度最小的路线，而不是路段数量最少的路线。

如果以后把权重改成预计行驶时间，那么同一套图算法求出的就是最快路线。当前仓库没有加入实时拥堵、红绿灯或时段变化，所以边权是静态的。

5. 什么是 OD

OD 是 Origin-Destination 的缩写：

- Origin：起点；
- Destination：终点。

一条 OD 记录表示“一次从某个起点到某个终点的出行需求”。Porto 原始数据来自出租车轨迹。项目取一条轨迹的首点和尾点，分别作为起点和终点，再把经纬度吸附到最近的道路节点。

“吸附”是指：轨迹坐标通常不会精确等于 OSM 道路节点坐标，因此要找一个足够近的道路节点代替它。项目保留吸附距离在 200 米以内的样本，得到 98,082 条可用 OD。

代码中的一条查询由 Query 表示，包含：

- query_id：查询编号；
- origin：起点节点编号；
- destination：终点节点编号；
- count：这条聚合需求出现多少次；
- timestamp：时间戳，用来按时间切分过去与未来。

6. 什么是最短路

从起点到终点可能存在很多路线。每条路线由若干条有向边首尾相接组成。把路线中所有边的权重加起来，得到路线总成本。总成本最小的可行路线就是最短路，其总成本叫最短距离。

项目只需要在线返回精确距离，不要求 GNN 输出路线节点序列。

“精确”意味着：压缩查询结果必须与直接在原图运行的最短路算法相同，允许的差异只有浮点数计算的极小误差，代码默认容差是 $1e-6$ 。

7. Dijkstra 是什么

Dijkstra 是一种经典最短路算法，适用于所有边权非负的图。本项目边长都大于零，因此可以使用它。

可以把 Dijkstra 想成从起点向外扩散的水波：

1. 起点距离为 0；
2. 每次确认当前已知距离最小、还没处理过的节点；
3. 尝试用这个节点改善其邻居的距离；
4. 直到确认终点，或再也没有可到达节点。

“尝试用当前节点改善邻居距离”叫作**松弛 (relaxation)**。

“确认一个节点的最短距离，不再需要改变它”叫作**展开或 settle**。本项目把展开节点数作为主要工作量指标：展开越少，通常代表算法做的确定性搜索工作越少。

8. 双向 Dijkstra 是什么

普通 Dijkstra 只从起点向终点搜索。双向 Dijkstra 同时进行：

- 正向搜索：从起点沿原图边前进；
- 反向搜索：从终点沿反向图寻找能够到达终点的节点。

两边在中间相遇后，往往不必搜索普通 Dijkstra 那么大的范围。

Porto 全量基线结果是：

方法	平均耗时	P95 耗时	平均展开节点	可达查询
普通	17.976	92.645	24,831.37	97,009 /
Dijkstra	ms	ms		98,082
双向	8.339 ms	37.341	10,741.70	97,009 /
Dijkstra		ms		98,082

P95 表示把查询耗时从小到大排列后，位于 95% 位置附近的值。它比平均值更能反映较慢查询的尾部情况。

9. “不可达” 是什么意思

有向道路图不保证任意两个节点都互相可达。某些起终点之间可能不存在任何合法的有向路线。代码用无穷大 `inf` 表示不可达距离。

正确性不仅要求可达查询的距离相同，也要求原图不可达时压缩图同样判断为不可达。

10. 离线与在线

本项目严格区分：

- **离线阶段：** 查询服务前做的预处理，例如生成区域、计算 shortcut、构建压缩图、

生成训练标签和训练模型；

- **在线阶段：** 真正接到一条 OD 后，运行压缩图双向 Dijkstra 并返回答案。

离线可以比较昂贵，因为它只需周期性执行；在线查询会重复很多次，所以每条查询节省的一点工作量累计起来可能很有价值。

但这不意味着离线成本可以无限大。项目仍记录预处理时间、shortcut 数和索引空间，并在后续版本研究更新成本能否由未来查询收益摊销。

11. 整个项目的五个阶段

阶段一：可信的图和评测底座

完成 Porto 道路图、OD 吸附、Dijkstra、双向 Dijkstra、全量查询明细、正确性检查。

阶段二：证明压缩确实可用

用随机区域和 OD 热点区域构建物化压缩图。结果保持 100% 正确，并减少了平均在线查询耗时，说明“区域压缩”这条工程路线成立。

阶段三：确定公平预算

扫描区域数量、区域大小和随机种子。最终建议把 512 节点区域和约 3.8 万至 4 万条 shortcut 作为第一版方法之间的公平预算参考。

阶段四：学习哪些区域值得压缩

第一版做过 GraphSAGE 节点评分，但消融发现主要收益来自手工 Proxy 和端点风险。当前第二版改为使用未来窗口的真实区域收益，并正式确定以 OD 条件化双向 NBFNet 为主模型，验证有向、查询条件化的需求传播是否真的有独立贡献。

阶段五：最终比较和扩展

在模型结构和预算冻结后，比较 NBFNet、MLP、随机、热点等方法，再做多随机种子、消融、跨时间和跨城市验证。当前尚未开始。

12. 到这里应该形成的心智模型

请先记住下面这条主线：

大量历史 OD

↓

判断哪些道路区域未来经常值得快速穿过

↓

离线把这些区域内部的精确距离压缩成 shortcut

↓

在线仍用精确双向 Dijkstra 查询

↓

距离答案不变，但希望搜索工作变少

下一章会具体解释“区域”“边界”“内部节点”“shortcut”和“物化压缩图”到底是什么。